cl0p Malware

Malware analysis report - Cl0p ransomware, Linux ELF variant

Analyzed sample	ELF32 executable named `sample` (1.19 MiB)
Target platform	Linux, Intel 386, 32-bit, little-endian (compiled with GCC 4.1.2 on Red Hat)
Malware family	Cl0p ransomware (Linux variant)
Evidence analyzed	Single binary sample, static analysis only (the sample was not executed)
Tools used	HashCalc, Detect It Easy (DIE), IDA Freeware
Overall result	Confirmed Cl0p ransomware: 7 targeted directories, hard-coded RC4 master key, `.C_I_0P` marker, ransom note `README_C_I_0P.TXT`

1. Executive Summary

A company was hit by a Cl0p ransomware attack and its critical data was encrypted. From my analyst workstation I examined the ransomware binary recovered from the incident to understand what it targets, how it encrypts, and how it communicates its demands to the victim.

The sample is a 32-bit Linux executable. Static analysis in IDA shows that it detaches itself as a background process, then loops forever calling a function named `do_heartbeat`. That function recursively searches seven directories for files to encrypt, using a hard-coded RC4 master key. For every processed file a companion key file ending in `.C_I_0P` is written, and a ransom note named `README_C_I_0P.TXT` is dropped that tells the victim to contact the attackers by email or through a chat on the Tor network. Any Linux server holding data in the targeted directories must be treated as compromised.

2. Scope and Evidence

The only evidence available is the ransomware binary itself, so every finding comes from static analysis. The sample was hashed with HashCalc, identified with Detect It Easy, and disassembled with IDA Freeware. No dynamic execution was performed.

3. Background: Cl0p Ransomware on Linux

Cl0p is a ransomware family that operates under a ransomware-as-a-service model and is best known for its Windows builds. This sample is a native Linux build, which means it targets servers and workloads rather than desktop users. Knowing that Linux builds of this family rely on a symmetric cipher with an embedded master key, and on a recursive file search started from a few fixed paths, defines what to look for in the disassembly: the function that starts the search, the list of start paths, the key material, the marker appended to processed files and the ransom note template.

4. Investigation Phases

Phase 1 - Sample Identification and Hashing

I started by fingerprinting the sample so it can be tracked and compared with other reports. In HashCalc I selected the file `ChallengeFile\sample\sample` and calculated MD5, SHA1, SHA256, RIPEMD160 and CRC32. The MD5 is `31e0439e6ef1dd29c0db6d96bac59446` and the SHA1 is `46b02cc186b85e11c3d59790c3a0bfd2ae1f82a5`.

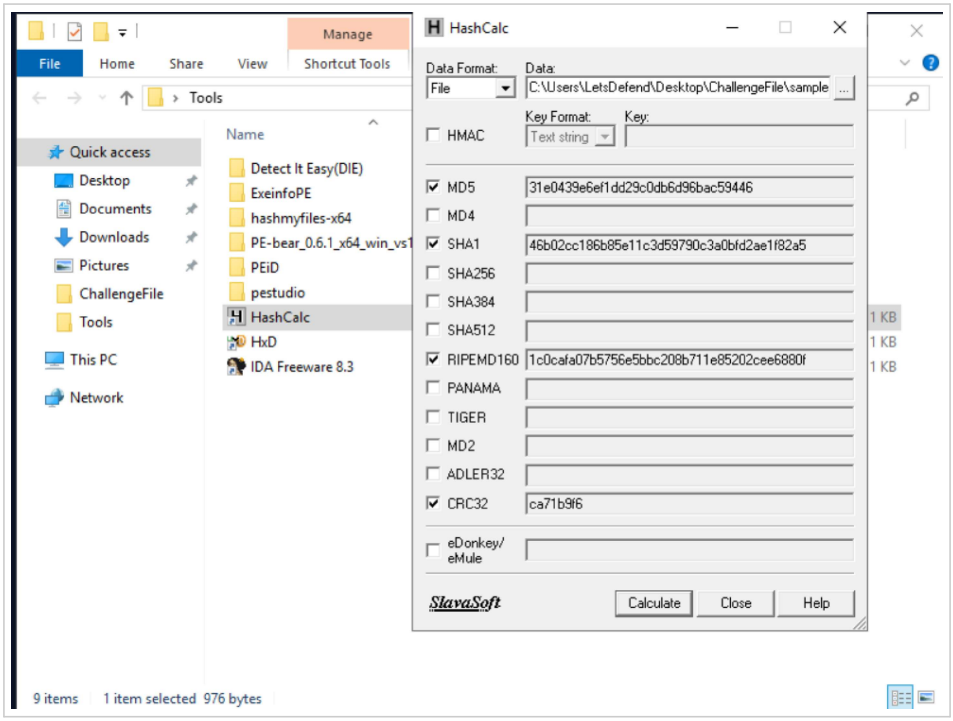


Figure 1. HashCalc open next to the analysis tools folder, showing the MD5, SHA1, RIPEMD160 and CRC32 values of the sample.

I then enabled the SHA256 algorithm and recalculated. The SHA256 value begins with `41eaa485b37de9a40c86b6d2eae7413db11b4e6a8256e` and is the primary identifier used for the indicator list.

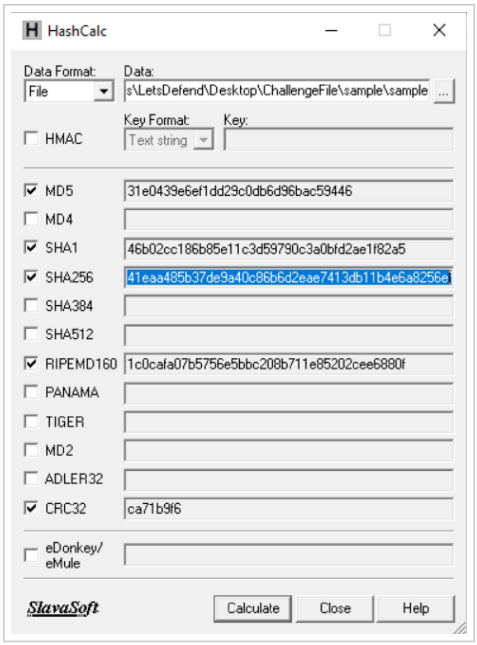


Figure 2. HashCalc with SHA256 enabled; the SHA256 field is selected.

Phase 2 - File Type and Packing Check

Before disassembling, I loaded the sample in Detect It Easy to see what kind of file it is and whether it is packed. DIE identifies an `ELF32` executable of 1.19 MiB, little-endian, 32-bit, architecture 386, type EXEC, for Red Hat Linux (ABI 2.6.9), built with GCC 4.1.2 (Red Hat 4.1.2-48) from C/C++ source. Nothing indicates a packer, so the code and strings can be read directly. The result also tells me this is a Linux ransomware and not a Windows binary.

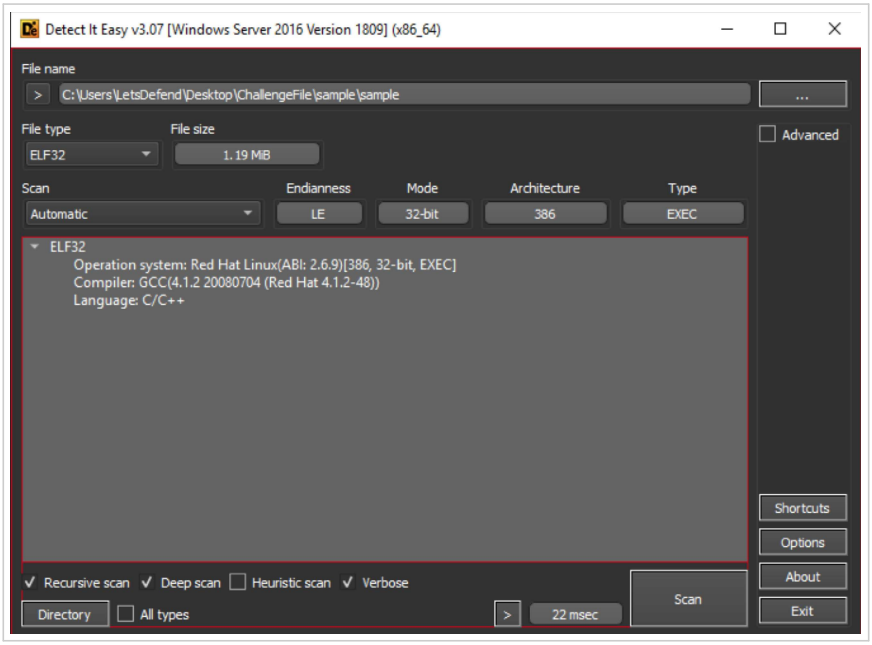


Figure 3. Detect It Easy identifying the sample as a 32-bit Linux ELF executable compiled with GCC.

Phase 3 - Disassembly and Program Flow

I opened the sample in IDA Freeware and let the initial autoanalysis finish. The function list contains descriptive names that outline the malware: `_rc4Init`, `_rc4`, `_rc4Full`, `CreateRadMe`, `Createkey`, `randx`, `EncrFile`, `find`, `do_heartbeat` and `main`. The RC4 routines already point to the cipher in use.

In the graph view of `main` the sample first closes file descriptors 0, 1 and 2 (standard input, output and error), a typical daemonization step. A string referenced just before it, `Could not change working directory to /`, belongs to the same routine. It then enters an infinite loop that calls `do_heartbeat` and then `sleep(0xFFFFFFFF)`, so the encryption routine is started from the main loop and the process stays alive in the background.

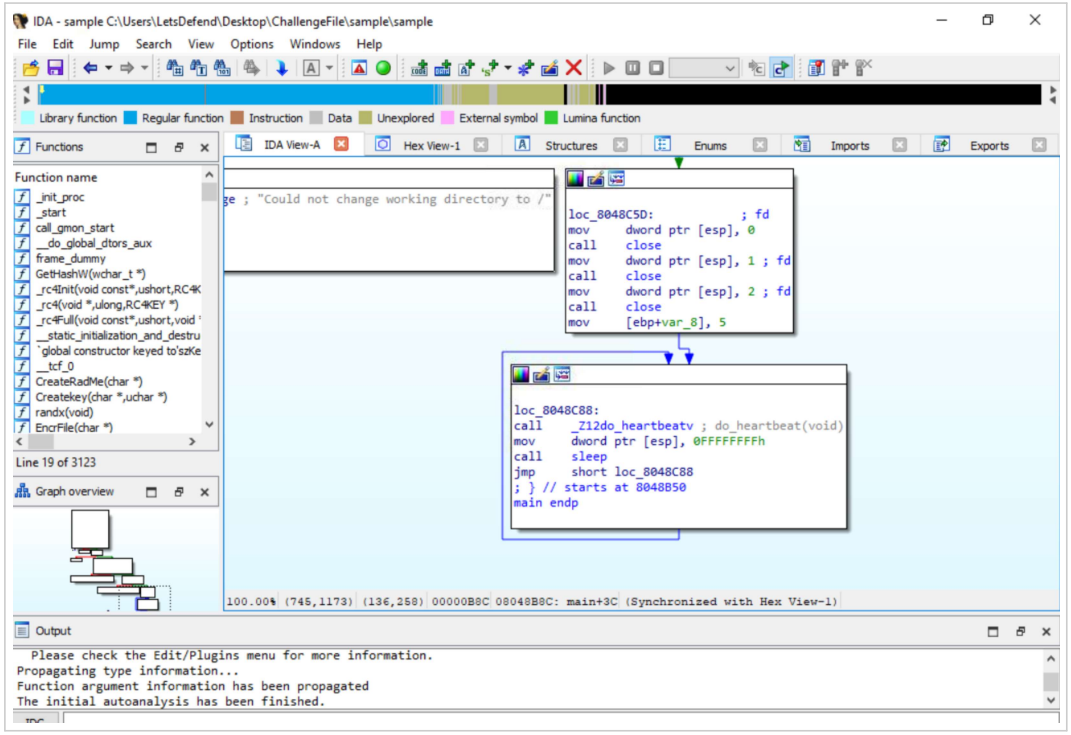


Figure 4. Graph view of main: standard descriptors are closed, then do_heartbeat is called inside an endless loop with a sleep.

Phase 4 - The do_heartbeat Function and Targeted Directories

I followed the call to `_Z12do_heartbeatv` (the mangled name of `do_heartbeat(void)`) from the loop in main. The cross-reference popup shows that the function is a public, bp-based procedure called from `main:loc_8048C88`.

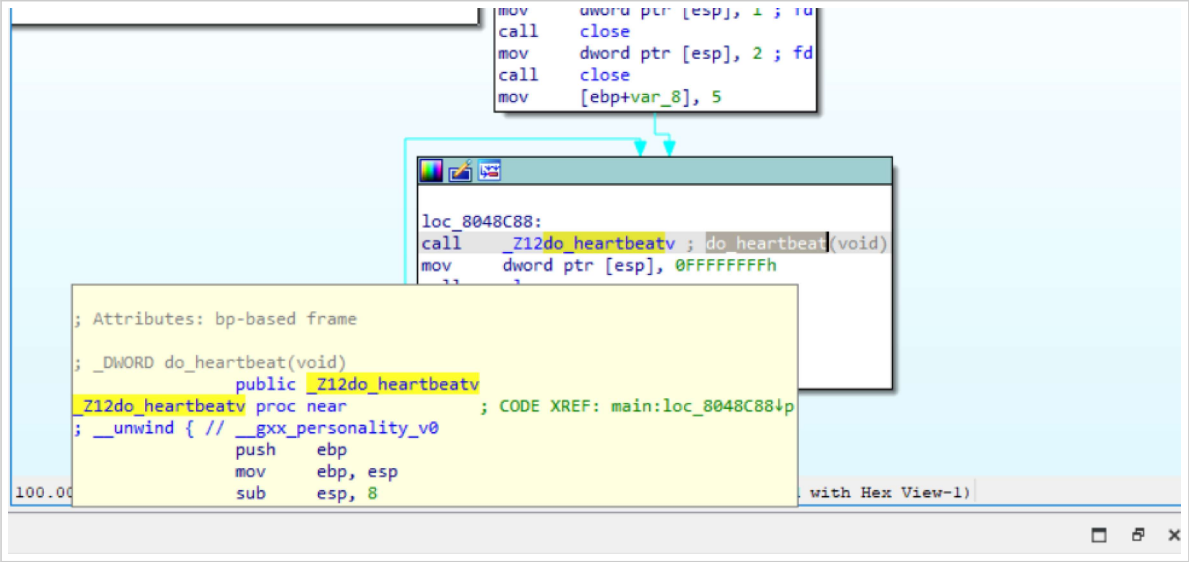


Figure 5. Preview of do_heartbeat(void) reached from the call inside main's loop.

Inside the function, the same pattern repeats: the string `"*.*"` is placed on the stack as the file pattern, a start path is loaded, and `find(char *, char const *)` is called. The start paths are `/opt`, `/u01`, `/u02`, `/u03`, `/u04`, `/home` and `/root`, which makes **seven targeted directories**. The pattern `*.*` means every file with an extension is a candidate. The paths cover application installs, the usual data mount points of database servers and all user home directories.

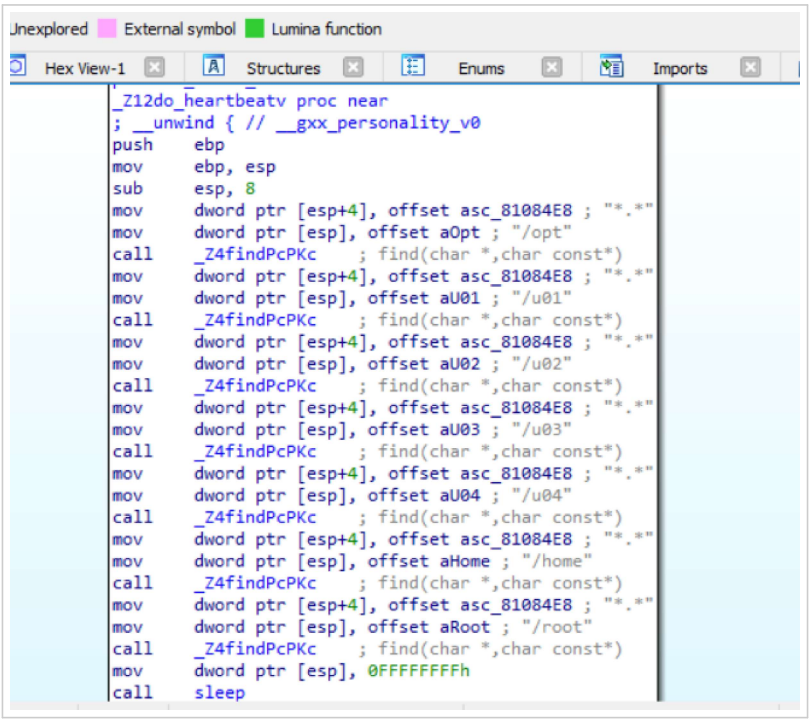


*Figure 6. do_heartbeat calling find() once per target directory with the *.* pattern.*

At the end of the function, after the last call to `find` and a `sleep`, the function returns. IDA marks the start of the function at address `0x08048AB0`, which is the offset of `do_heartbeat` in the binary.

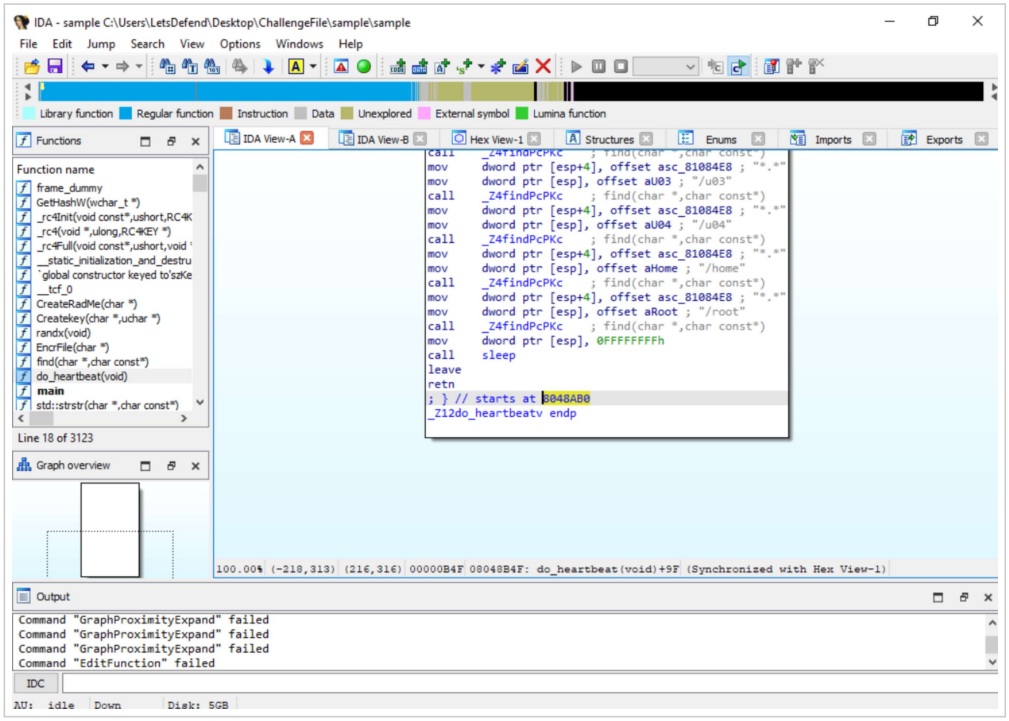


Figure 7. End of do_heartbeat showing the last search target (/root), the closing sleep and the function start address 8048AB0.

Phase 5 - Hard-coded Encryption Key

To find the key, I went back to the beginning of `main`. After calling `time` and `srandom` to seed the random generator, the code loads the address of a global buffer named `szKeyKey` and the address of a long string, then calls `memcpy` with a length of `0x42` bytes. The string, `aJfkdskfku2ir32y7432uroduw8y7318i9018ure...` in IDA, is therefore copied into `szKeyKey` at startup, and a `fork` call follows.

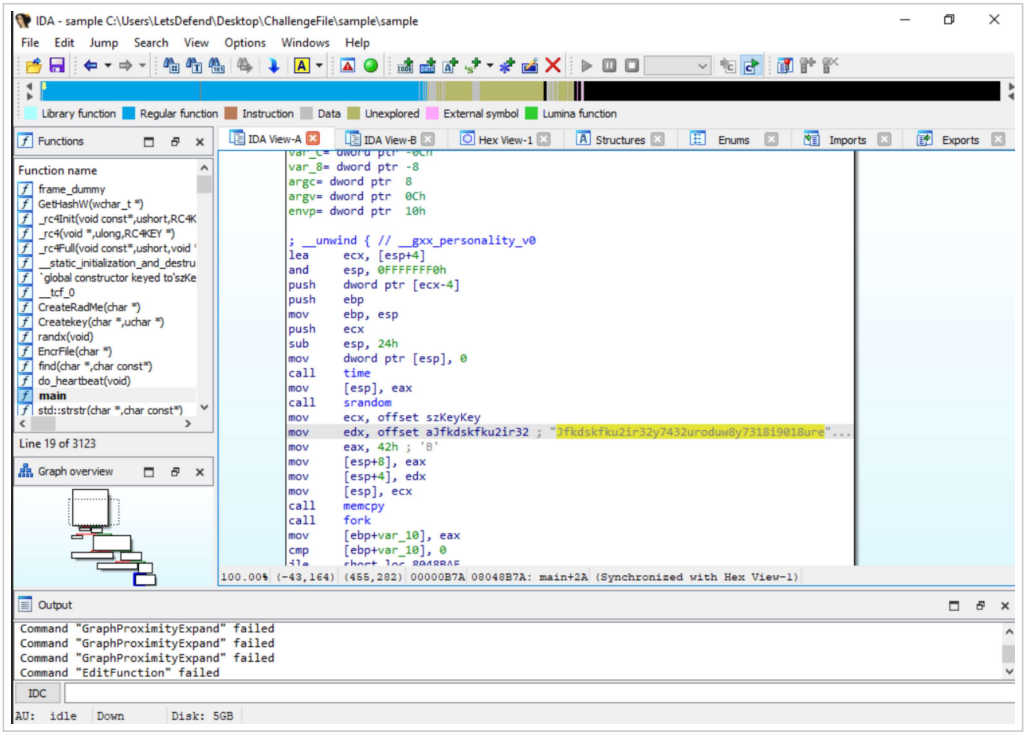


Figure 8. Start of main: the hard-coded string is copied into szKeyKey with memcpy before the process forks.

Following the string to the read-only data section (`.rodata:08108514`) shows the complete key, referenced from `main+2A`. The same area contains the messages used while daemonizing, such as `Successfully started daemon-name` and `Could not generate session ID for child process`, which confirm that the sample forks

and detaches from the terminal.

Master key: `Jfkdskfku2ir32y7432uroduw8y7318i9018urewfdsZ2Oaifwuieh~~cudsffdsd`

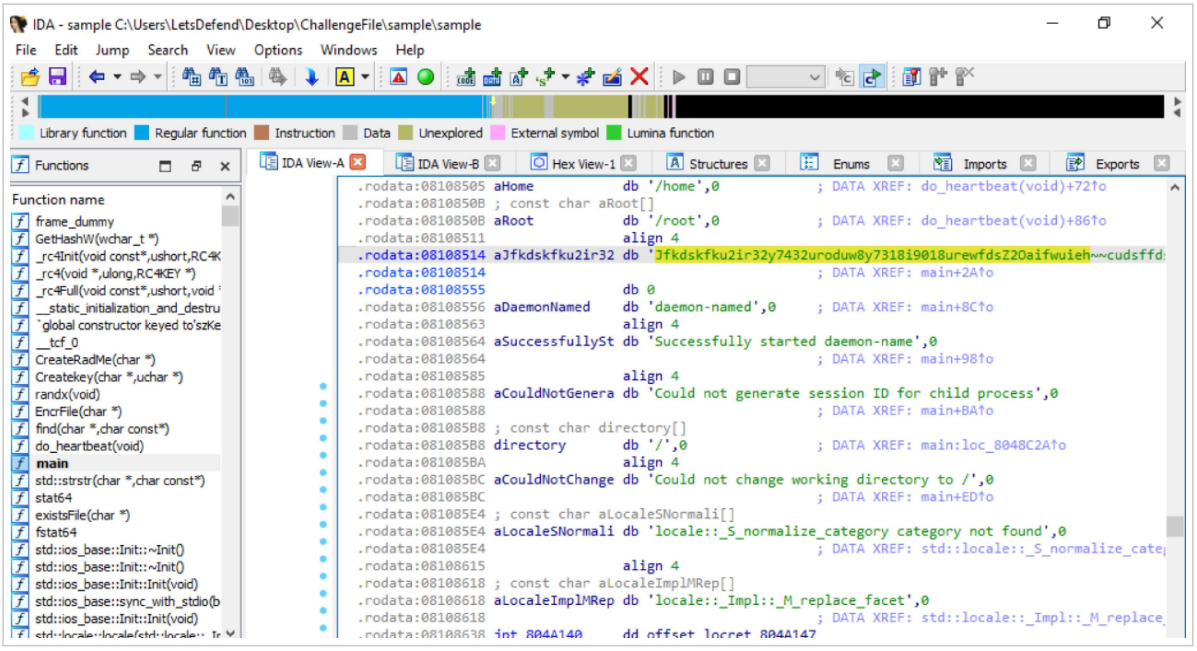


Figure 9. The full key in .rodata, with its cross-reference to main+2A.

Phase 6 - Key Files and the .C_I_0P Marker

Next I opened `Createkey(char *, uchar *)`. It builds a file name with `sprintf` using the format string `"%s.C_I_0P"`, opens it with `open64` (flags `0x42`, mode `0x1B4`), writes `0x100` bytes and closes it. In other words, for every victim file the malware creates a companion file whose name is the original path plus `.C_I_0P`, holding the per-file key material. The same string is referenced from `find` (as `aCI0p`), which suggests that the search routine uses it to recognize files that have already been processed.

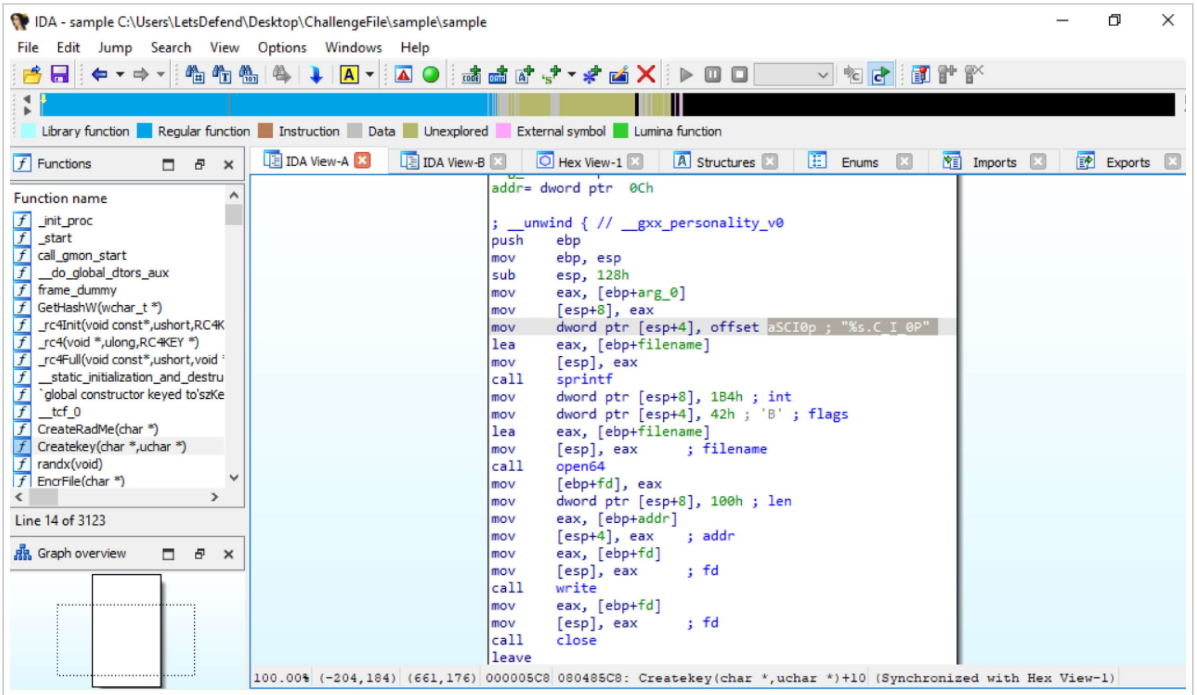


Figure 10. Createkey building the file name with the format string %s.C_I_0P and writing 0x100 bytes to it.

Phase 7 - Ransom Note

The function `CreateRadMe(char *)` takes a folder as its argument and creates the ransom note inside it. In the data section, the format string `"%s/README_C_I_0P.TXT"` gives the note's file name, referenced from `CreateRadMe+31`.

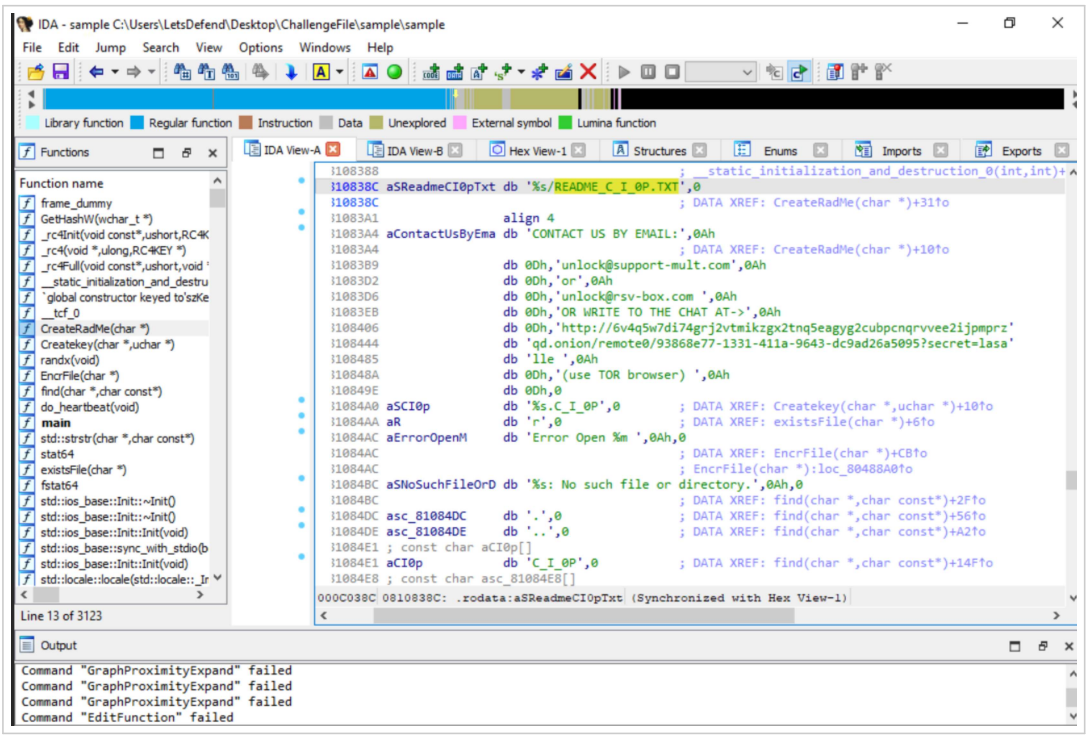


Figure 11. The ransom note path format string %s/README_C_I_0P.TXT, referenced by CreateRadMe.

The text that follows is the body of the note. It tells the victim to contact the attackers by email at two addresses, `unlock@support-mult[.]com` and `unlock@rsv-box[.]com`, or to use a chat hosted on a Tor hidden service (the note explicitly asks the victim to use the Tor browser). The chat link contains a victim-specific identifier and secret.

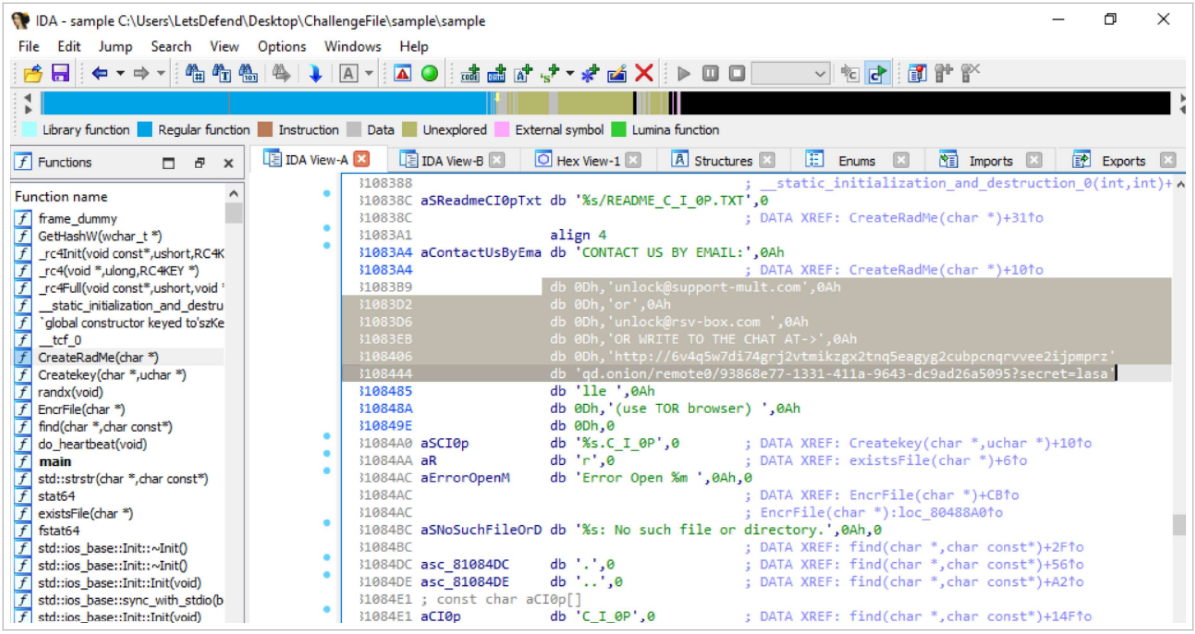


Figure 12. Ransom note body: contact email addresses and the Tor chat link, with the instruction to use the Tor browser.

5. Execution Flow of the Sample

Step	Behavior	Where observed
1	main seeds the random generator with time() and srandom()	Figure 8
2	The hard-coded master key is copied into szKeyKey (memcpy, 0x42 bytes)	Figure 8
3	The process forks and detaches (session creation, working directory set to /)	Figures 4, 8, 9
4	Standard input, output and error are closed	Figure 4
5	Endless loop: do_heartbeat() followed by sleep(0xFFFFFFFF)	Figure 4
6	do_heartbeat calls find() with *.* on seven directories	Figures 6, 7
7	Matching files are encrypted (EncrFile) using RC4 with the master key	Function list, Figures 3-4
8	Createkey writes the companion key file named <file>.C_I_0P	Figure 10
9	CreateRadMe drops README_C_I_0P.TXT with the contact details	Figures 11, 12

The order is reconstructed from the disassembly of the functions and their cross-references; the sample was not executed.

6. MITRE ATT&CK Mapping

Technique	ID	Observed activity
Data Encrypted for Impact	T1486	EncrFile and the RC4 routines encrypt files under the targeted directories
File and Directory Discovery	T1083	find() recursively searches seven directories with the *.* pattern
Financial Theft (extortion)	T1657	Ransom note directs the victim to pay and negotiate by email or Tor chat

7. Indicators of Compromise

Type	Value	Notes
MD5	`31e0439e6ef1dd29c0db6d96bac59446`	Hash of the sample
SHA1	`46b02cc186b85e11c3d59790c3a0bfd2ae1f82a5`	Hash of the sample
SHA256	`41eaa485b37de9a40c86b6d2eae7413db11b4e6a8256e...`	Hash of the sample (value cut off in the capture)
File type	ELF32, Intel 386, GCC 4.1.2 (Red Hat)	Linux ransomware binary
File marker	`.C_I_0P`	Suffix of the companion key files written by Createkey
Ransom note	`README_C_I_0P.TXT`	Dropped in the folders that were processed
Encryption key	`Jfkdskfku2ir32y7432uroduw8y7318i9018urewfdsZ2Oaifwuieh~~cudsffdsd`	Hard-coded RC4 master key (szKeyKey)
Email	`unlock@support-mult[.]com`	Attacker contact in the note
Email	`unlock@rsv-box[.]com`	Attacker contact in the note
Tor chat	`hxxp://6v4q5w7di74grj2vtmikzgx2tnq5eagyg2cubpcnqrvvee2ijpmprzqd[.]onion/remote0/...`	Attacker chat link (identifier and secret omitted)
Strings	`do_heartbeat`, `CreateRadMe`, `Createkey`, `EncrFile`, `szKeyKey`	Useful for YARA or string-based detection
Target paths	`/opt` `/u01` `/u02` `/u03` `/u04` `/home` `/root`	Directories searched for files to encrypt

8. Impact Assessment

The sample encrypts every file with an extension under the application, database-mount and user directories of a Linux server. Because `/u01` to `/u04` are common database and data volumes and `/home` and `/root` hold user data and administrator material, business data, database files and user files must be considered lost or exposed. The process runs as a detached background loop that never exits, so it keeps scanning and re-processing the directories until it is killed.

9. Recommendations

- Isolate any affected Linux host from the network and terminate the detached process before it encrypts further data.
- Search the fleet for the hashes, the file name `README_C_I_0P.TXT` and files ending in `.C_I_0P` to find other affected hosts.
- Block the contact email domains and Tor access at the perimeter, and monitor for the strings and key listed in the indicator table.
- Restore from clean, offline or immutable backups; do not rely on backups stored in the targeted directories.
- Check whether a public decryptor exists for this Linux variant, since the file-key handling in early Linux builds of this family has been reported as flawed, before considering any payment.
- Reduce exposure: patch internet-facing services, restrict administrative access and apply least privilege on data mounts.

- Add detections for a process that closes standard descriptors, forks and then reads or rewrites large numbers of files under /opt, /u0x, /home and /root.

10. Conclusion

Static analysis of the sample confirms a Linux build of the Cl0p ransomware. It daemonizes, repeatedly calls `do_heartbeat` at address `0x08048AB0`, encrypts files in seven directories with a hard-coded RC4 key, writes `.C_I_0P` companion key files and leaves a `README_C_I_0P.TXT` note pointing to two email addresses and a Tor chat. The indicators above can be used to hunt for the sample and its artifacts, and the recommendations focus on containment, recovery from clean backups and detection of the behavior.